

359

`ivec: size: 51 capacity: 100`

这表明 `vector` 的实现采用的策略似乎是在每次需要分配新内存空间时将当前容量翻倍。

可以调用 `shrink_to_fit` 来要求 `vector` 将超出当前大小的多余内存退回给系统：

```
ivec.shrink_to_fit(); // 要求归还内存
// size 应该未改变；capacity 的值依赖于具体实现
cout << "ivec: size: " << ivec.size()
      << " capacity: " << ivec.capacity() << endl;
```

调用 `shrink_to_fit` 只是一个请求，标准库并不保证退还内存。



每个 `vector` 实现都可以选择自己的内存分配策略。但是必须遵守的一条原则是：只有当迫不得已时才可以分配新的内存空间。

只有在执行 `insert` 操作时 `size` 与 `capacity` 相等，或者调用 `resize` 或 `reserve` 时给定的大小超过当前 `capacity`，`vector` 才可能重新分配内存空间。会分配多少超过给定容量的额外空间，取决于具体实现。

虽然不同的实现可以采用不同的分配策略，但所有实现都应遵循一个原则：确保用 `push_back` 向 `vector` 添加元素的操作有高效率。从技术角度说，就是通过在一个初始为空的 `vector` 上调用 n 次 `push_back` 来创建一个 n 个元素的 `vector`，所花费的时间不能超过 n 的常数倍。

9.4 节练习

练习 9.35: 解释一个 `vector` 的 `capacity` 和 `size` 有何区别。

练习 9.36: 一个容器的 `capacity` 可能小于它的 `size` 吗？

练习 9.37: 为什么 `list` 或 `array` 没有 `capacity` 成员函数？

练习 9.38: 编写程序，探究在你的标准库实现中，`vector` 是如何增长的。

练习 9.39: 解释下面程序片段做了什么：

```
vector<string> svec;
svec.reserve(1024);
string word;
while (cin >> word)
    svec.push_back(word);
svec.resize(svec.size()+svec.size()/2);
```

练习 9.40: 如果上一题中的程序读入了 256 个词，在 `resize` 之后容器的 `capacity` 可能是多少？如果读入了 512 个、1000 个或 1048 个词呢？

360

9.5 额外的 `string` 操作

除了顺序容器共同的操作之外，`string` 类型还提供了一些额外的操作。这些操作中的大部分要么是提供 `string` 类和 C 风格字符数组之间的相互转换，要么是增加了允许我们用下标代替迭代器的版本。

标准库 `string` 类型定义了大量函数。幸运的是，这些函数使用了重复的模式。由于函数过多，本节初次阅读可能令人心烦，因此读者可能希望快速浏览本节。当你了解 `string` 支持哪些类型的操作后，就可以在需要使用一个特定操作时回过头来仔细阅读。

9.5.1 构造 string 的其他方法



除了我们在 3.2.1 节（第 76 页）已经介绍过的构造函数，以及与其他顺序容器相同的构造函数（参见表 9.3，第 299 页）外，`string` 类型还支持另外三个构造函数，如表 9.11 所示。

表 9.11: 构造 string 的其他方法	
n、len2 和 pos2 都是无符号值	
<code>string s(cp,n)</code>	s 是 cp 指向的数组中前 n 个字符的拷贝。此数组至少应该包含 n 个字符
<code>string s(s2,pos2)</code>	s 是 string s2 从下标 pos2 开始的字符的拷贝。若 <code>pos2>s2.size()</code> ，构造函数的行为未定义
<code>string s(s2,pos2,len2)</code>	s 是 string s2 从下标 pos2 开始 len2 个字符的拷贝。若 <code>pos2>s2.size()</code> ，构造函数的行为未定义。不管 len2 的值是多少，构造函数至多拷贝 <code>s2.size()-pos2</code> 个字符

这些构造函数接受一个 `string` 或一个 `const char*` 参数，还接受（可选的）指定拷贝多少个字符的参数。当我们传递给它们的是一个 `string` 时，还可以给定一个下标来指出从哪里开始拷贝：

```
const char *cp = "Hello World!!!";    // 以空字符结束的数组
char noNull[] = {'H', 'i'};           // 不是以空字符结束
string s1(cp); // 拷贝 cp 中的字符直到遇到空字符; s1 == "Hello World!!!"
string s2(noNull,2); // 从 noNull 拷贝两个字符; s2 == "Hi"
string s3(noNull); // 未定义: noNull 不是以空字符结束
string s4(cp + 6, 5); // 从 cp[6] 开始拷贝 5 个字符; s4 == "World"
string s5(s1, 6, 5); // 从 s1[6] 开始拷贝 5 个字符; s5 == "World"
string s6(s1, 6); // 从 s1[6] 开始拷贝, 直至 s1 末尾; s6 == "World!!!"
string s7(s1,6,20); // 正确, 只拷贝到 s1 末尾; s7 == "World!!!"
string s8(s1, 16); // 抛出一个 out_of_range 异常
```

通常当我们从一个 `const char*` 创建 `string` 时，指针指向的数组必须以空字符结尾，拷贝操作遇到空字符时停止。如果我们还传递给构造函数一个计数值，数组就不必以空字符结尾。如果我们未传递计数值且数组也未以空字符结尾，或者给定计数值大于数组大小，则构造函数的行为是未定义的。

361

当从一个 `string` 拷贝字符时，我们可以提供一个可选的开始位置和一个计数值。开始位置必须小于或等于给定的 `string` 的大小。如果位置大于 `size`，则构造函数抛出一个 `out_of_range` 异常（参见 5.6 节，第 173 页）。如果我们传递了一个计数值，则从给定位置开始拷贝这么多个字符。不管我们要求拷贝多少个字符，标准库最多拷贝到 `string` 结尾，不会更多。

substr 操作

`substr` 操作（参见表 9.12）返回一个 `string`，它是原始 `string` 的一部分或全部的拷贝。可以传递给 `substr` 一个可选的开始位置和计数值：

```

string s("hello world");
string s2 = s.substr(0, 5);           // s2 = hello
string s3 = s.substr(6);             // s3 = world
string s4 = s.substr(6, 11);         // s3 = world
string s5 = s.substr(12);            // 抛出一个 out_of_range 异常

```

如果开始位置超过了 `string` 的大小, 则 `substr` 函数抛出一个 `out_of_range` 异常(参见 5.6 节, 第 173 页)。如果开始位置加上计数值大于 `string` 的大小, 则 `substr` 会调整计数值, 只拷贝到 `string` 的末尾。

表 9.12: 子字符串操作

<code>s.substr(pos, n)</code>	返回一个 <code>string</code> , 包含 <code>s</code> 中从 <code>pos</code> 开始的 <code>n</code> 个字符的拷贝。 <code>pos</code> 的默认值为 0。 <code>n</code> 的默认值为 <code>s.size() - pos</code> , 即拷贝从 <code>pos</code> 开始的所有字符
-------------------------------	--

9.5.1 节练习

练习 9.41: 编写程序, 从一个 `vector<char>` 初始化一个 `string`。

练习 9.42: 假定你希望每次读取一个字符存入一个 `string` 中, 而且知道最少需要读取 100 个字符, 应该如何提高程序的性能?

9.5.2 改变 `string` 的其他方法

`string` 类型支持顺序容器的赋值运算符以及 `assign`、`insert` 和 `erase` 操作(参见 9.2.5 节, 第 302 页; 9.3.1 节, 第 306 页; 9.3.3 节, 第 311 页)。除此之外, 它还定义了额外的 `insert` 和 `erase` 版本。

除了接受迭代器的 `insert` 和 `erase` 版本外, `string` 还提供了接受下标的版本。下标指出了开始删除的位置, 或是 `insert` 到给定值之前的位置:

```

s.insert(s.size(), 5, '!'); // 在 s 末尾插入 5 个感叹号
s.erase(s.size() - 5, 5);  // 从 s 删除最后 5 个字符

```

362

标准库 `string` 类型还提供了接受 C 风格字符数组的 `insert` 和 `assign` 版本。例如, 我们可以将以空字符结尾的字符数组 `insert` 到或 `assign` 给一个 `string`:

```

const char *cp = "Stately, plump Buck";
s.assign(cp, 7);           // s == "Stately"
s.insert(s.size(), cp + 7); // s == "Stately, plump Buck"

```

此处我们首先通过调用 `assign` 替换 `s` 的内容。我们赋予 `s` 的是从 `cp` 指向的地址开始的 7 个字符。要求赋值的字符数必须小于或等于 `cp` 指向的数组中的字符数(不包括结尾的空字符)。

接下来在 `s` 上调用 `insert`, 我们的意图是将字符插入到 `s[size()]` 处(不存在的)元素之前的位置。在此例中, 我们将 `cp` 开始的 7 个字符(至多到结尾空字符之前)拷贝到 `s` 中。

我们也可以指定将来自其他 `string` 或子字符串的字符插入到当前 `string` 中或赋予当前 `string`:

```

string s = "some string", s2 = "some other string";
s.insert(0, s2); // 在 s 中位置 0 之前插入 s2 的拷贝

```

```
// 在 s[0] 之前插入 s2 中 s2[0] 开始的 s2.size() 个字符
s.insert(0, s2, 0, s2.size());
```

append 和 replace 函数

string 类定义了两个额外的成员函数：append 和 replace，这两个函数可以改变 string 的内容。表 9.13 描述了这两个函数的功能。append 操作是在 string 末尾进行插入操作的一种简写形式：

```
string s("C++ Primer"), s2 = s; // 将 s 和 s2 初始化为 "C++ Primer"
s.insert(s.size(), " 4th Ed."); // s == "C++ Primer 4th Ed."
s2.append(" 4th Ed."); // 等价方法：将 " 4th Ed." 追加到 s2; s == s2
```

replace 操作是调用 erase 和 insert 的一种简写形式：

```
// 将 "4th" 替换为 "5th" 的等价方法
s.erase(11, 3); // s == "C++ Primer Ed."
s.insert(11, "5th"); // s == "C++ Primer 5th Ed."
// 从位置 11 开始，删除 3 个字符并插入 "5th"
s2.replace(11, 3, "5th"); // 等价方法：s == s2
```

此例中调用 replace 时，插入的文本恰好与删除的文本一样长。这不是必须的，可以插入一个更长或更短的 string：

```
s.replace(11, 3, "Fifth"); // s == "C++ Primer Fifth Ed."
```

在此调用中，删除了 3 个字符，但在其位置插入了 5 个新字符。

表 9.13: 修改 string 的操作

363

s.insert(<i>pos</i> , <i>args</i>)	在 <i>pos</i> 之前插入 <i>args</i> 指定的字符。 <i>pos</i> 可以是一个下标或一个迭代器。接受下标的版本返回一个指向 s 的引用；接受迭代器的版本返回指向第一个插入字符的迭代器
s.erase(<i>pos</i> , <i>len</i>)	删除从位置 <i>pos</i> 开始的 <i>len</i> 个字符。如果 <i>len</i> 被省略，则删除从 <i>pos</i> 开始直至 s 末尾的所有字符。返回一个指向 s 的引用
s.assign(<i>args</i>)	将 s 中的字符替换为 <i>args</i> 指定的字符。返回一个指向 s 的引用
s.append(<i>args</i>)	将 <i>args</i> 追加到 s。返回一个指向 s 的引用
s.replace(<i>range</i> , <i>args</i>)	删除 s 中范围 <i>range</i> 内的字符，替换为 <i>args</i> 指定的字符。 <i>range</i> 或者是一个下标和一个长度，或者是一对指向 s 的迭代器。返回一个指向 s 的引用
<i>args</i> 可以是下列形式之一；append 和 assign 可以使用所有形式。	
str 不能与 s 相同，迭代器 b 和 e 不能指向 s。	
str	字符串 str
str, pos, len	str 中从 pos 开始最多 len 个字
cp, len	从 cp 指向的字符数组的前（最多）len 个字符
cp	cp 指向的以空字符结尾的字符数组
n, c	n 个字符 c
b, e	迭代器 b 和 e 指定的范围内的字符
初始化列表	花括号包围的，以逗号分隔的字符列表

续表

replace 和 insert 所允许的 args 形式依赖于 range 和 pos 是如何指定的。				
replace (pos, len, args)	replace (b, e, args)	insert (pos, args)	insert (iter, args)	args 可以是
是	是	是	否	str
是	否	是	否	str, pos, len
是	是	是	否	cp, len
是	是	否	否	cp
是	是	是	是	n, c
否	是	否	是	b2, e2
否	是	否	是	初始化列表

改变 string 的多种重载函数

表 9.13 列出的 append、assign、insert 和 replace 函数有多个重载版本。根据我们如何指定要添加的字符和 string 中被替换的部分，这些函数的参数有不同版本。幸运的是，这些函数有共同的接口。

assign 和 append 函数无须指定要替换 string 中哪个部分：assign 总是替换 string 中的所有内容，append 总是将新字符追加到 string 末尾。

364

replace 函数提供了两种指定删除元素范围的方式。可以通过一个位置和一个长度来指定范围，也可以通过一个迭代器范围来指定。insert 函数允许我们用两种方式指定插入点：用一个下标或一个迭代器。在两种情况下，新元素都会插入到给定下标（或迭代器）之前的位置。

可以用好几种方式来指定要添加到 string 中的字符。新字符可以来自于另一个 string，来自于一个字符指针（指向的字符数组），来自于一个花括号包围的字符列表，或者是一个字符和一个计数值。当字符来自于一个 string 或一个字符指针时，我们可以传递一个额外的参数来控制是拷贝部分还是全部字符。

并不是每个函数都支持所有形式的参数。例如，insert 就不支持下标和初始化列表参数。类似的，如果我们希望用迭代器指定插入点，就不能用字符指针指定新字符的来源。

9.5.2 节练习

练习 9.43: 编写一个函数，接受三个 string 参数 s、oldVal 和 newVal。使用迭代器及 insert 和 erase 函数将 s 中所有 oldVal 替换为 newVal。测试你的程序，用它替换通用的简写形式，如，将"tho"替换为"though"，将"thru"替换为"through"。

练习 9.44: 重写上一题的函数，这次使用一个下标和 replace。

练习 9.45: 编写一个函数，接受一个表示名字的 string 参数和两个分别表示前缀（如"Mr."或"Ms."）和后缀（如"Jr."或"III"）的字符串。使用迭代器及 insert 和 append 函数将前缀和后缀添加到给定的名字中，将生成的新 string 返回。

练习 9.46: 重写上一题的函数，这次使用位置和长度来管理 string，并只使用 insert。

9.5.3 string 搜索操作



string 类提供了 6 个不同的搜索函数，每个函数都有 4 个重载版本。表 9.14 描述了这些搜索成员函数及其参数。每个搜索操作都返回一个 `string::size_type` 值，表示匹配发生位置的下标。如果搜索失败，则返回一个名为 `string::npos` 的 static 成员（参见 7.6 节，第 268 页）。标准库将 `npos` 定义为一个 `const string::size_type` 类型，并初始化为值-1。由于 `npos` 是一个 unsigned 类型，此初始值意味着 `npos` 等于任何 string 最大的可能大小（参见 2.1.2 节，第 32 页）。



string 搜索函数返回 `string::size_type` 值，该类型是一个 unsigned 类型。因此，用一个 `int` 或其他带符号类型来保存这些函数的返回值不是一个好主意（参见 2.1.2 节，第 33 页）。

`find` 函数完成最简单的搜索。它查找参数指定的字符串，若找到，则返回第一个匹配位置的下标，否则返回 `npos`：

```
string name("AnnaBelle");
auto pos1 = name.find("Anna"); // pos1 == 0
```

365

这段程序返回 0，即子字符串"Anna"在"AnnaBelle"中第一次出现的下标。

搜索（以及其他 string 操作）是大小写敏感的。当在 string 中查找子字符串时，要注意大小写：

```
string lowercase("annabelle");
pos1 = lowercase.find("Anna"); // pos1 == npos
```

这段代码会将 `pos1` 置为 `npos`，因为 Anna 与 anna 不匹配。

一个更复杂一些的问题是查找与给定字符串中任何一个字符匹配的位置。例如，下面代码定位 `name` 中的第一个数字：

```
string numbers("0123456789"), name("r2d2");
// 返回 1，即，name 中第一个数字的下标
auto pos = name.find_first_of(numbers);
```

如果是要搜索第一个不在参数中的字符，我们应该调用 `find_first_not_of`。例如，为了搜索一个 string 中第一个非数字字符，可以这样做：

```
string dept("03714p3");
// 返回 5——字符'p'的下标
auto pos = dept.find_first_not_of(numbers);
```

表 9.14: string 搜索操作	
搜索操作返回指定字符出现的下标，如果未找到则返回 <code>npos</code> 。	
<code>s.find(args)</code>	查找 <code>s</code> 中 <code>args</code> 第一次出现的位置
<code>s.rfind(args)</code>	查找 <code>s</code> 中 <code>args</code> 最后一次出现的位置
<code>s.find_first_of(args)</code>	在 <code>s</code> 中查找 <code>args</code> 中任何一个字符第一次出现的位置。
<code>s.find_last_of(args)</code>	在 <code>s</code> 中查找 <code>args</code> 中任何一个字符最后一次出现的位置
<code>s.find_first_not_of(args)</code>	在 <code>s</code> 中查找第一个不在 <code>args</code> 中的字符
<code>s.find_last_not_of(args)</code>	在 <code>s</code> 中查找最后一个不在 <code>args</code> 中的字符

续表

args 必须是以下形式之一

<code>c, pos</code>	从 <code>s</code> 中位置 <code>pos</code> 开始查找字符 <code>c</code> 。 <code>pos</code> 默认为 0
<code>s2, pos</code>	从 <code>s</code> 中位置 <code>pos</code> 开始查找字符串 <code>s2</code> 。 <code>pos</code> 默认为 0
<code>cp, pos</code>	从 <code>s</code> 中位置 <code>pos</code> 开始查找指针 <code>cp</code> 指向的以空字符结尾的 C 风格字符串。 <code>pos</code> 默认为 0
<code>cp, pos, n</code>	从 <code>s</code> 中位置 <code>pos</code> 开始查找指针 <code>cp</code> 指向的数组的前 <code>n</code> 个字符。 <code>pos</code> 和 <code>n</code> 无默认值

指定在哪里开始搜索

我们可以传递给 `find` 操作一个可选的开始位置。这个可选的参数指出从哪个位置开始进行搜索。默认情况下，此位置被置为 0。一种常见的程序设计模式是用这个可选参数在字符串中循环地搜索子字符串出现的所有位置：

366

```
string::size_type pos = 0;
// 每步循环查找 name 中下一个数
while ((pos = name.find_first_of(numbers, pos))
        != string::npos) {
    cout << "found number at index: " << pos
          << " element is " << name[pos] << endl;
    ++pos; // 移动到下一个字符
}
```

`while` 的循环条件将 `pos` 重置为从 `pos` 开始遇到的第一个数字的下标。只要 `find_first_of` 返回一个合法下标，我们就打印当前结果并递增 `pos`。

如果我们忽略了递增 `pos`，循环就永远也不会终止。为了搞清楚原因，考虑如果不做递增运算会发生什么。在第二步循环中，我们从 `pos` 指向的字符开始搜索。这个字符是一个数字，因此 `find_first_of` 会（重复地）返回 `pos`！

逆向搜索

到现在为止，我们已经用过的 `find` 操作都是由左至右搜索。标准库还提供了类似的，但由右至左搜索的操作。`rfind` 成员函数搜索最后一个匹配，即子字符串最靠右的出现位置：

```
string river("Mississippi");
auto first_pos = river.find("is"); // 返回 1
auto last_pos = river.rfind("is"); // 返回 4
```

`find` 返回下标 1，表示第一个“is”的位置，而 `rfind` 返回下标 4，表示最后一个“is”的位置。

类似的，`find_last` 函数的功能与 `find_first` 函数相似，只是它们返回最后一个而不是第一个匹配：

- `find_last_of` 搜索与给定 `string` 中任何一个字符匹配的最后一个字符。
- `find_last_not_of` 搜索最后一个不出现在给定 `string` 中的字符。

每个操作都接受一个可选的第二参数，可用来指出从什么位置开始搜索。

9.5.3 节练习

练习 9.47: 编写程序, 首先查找 string "ab2c3d7R4E6"中的每个数字字符, 然后查找其中每个字母字符。编写两个版本的程序, 第一个要使用 find_first_of, 第二个要使用 find_first_not_of。

练习 9.48: 假定 name 和 numbers 的定义如 325 页所示, numbers.find(name) 返回什么?

练习 9.49: 如果一个字母延伸到中线之上, 如 d 或 f, 则称其有上出头部分 (ascender)。如果一个字母延伸到中线之下, 如 p 或 g, 则称其有下出头部分 (descender)。编写程序, 读入一个单词文件, 输出最长的既不包含上出头部分, 也不包含下出头部分的单词。

9.5.4 compare 函数

除了关系运算符外 (参见 3.2.2 节, 第 79 页), 标准库 string 类型还提供了一组 compare 函数, 这些函数与 C 标准库的 strcmp 函数 (参见 3.5.4 节, 第 109 页) 很相似。类似 strcmp, 根据 s 是等于、大于还是小于参数指定的字符串, s.compare 返回 0、正数或负数。

如表 9.15 所示, compare 有 6 个版本。根据我们是要比较两个 string 还是一个 string 与一个字符数组, 参数各有不同。在这两种情况下, 都可以比较整个或部分字符串。

表 9.15: s.compare 的几种参数形式	
s2	比较 s 和 s2
pos1, n1, s2	将 s 中从 pos1 开始的 n1 个字符与 s2 进行比较
pos1, n1, s2, pos2, n2	将 s 中从 pos1 开始的 n1 个字符与 s2 中从 pos2 开始的 n2 个字符进行比较
cp	比较 s 与 cp 指向的以空字符结尾的字符数组
pos1, n1, cp	将 s 中从 pos1 开始的 n1 个字符与 cp 指向的以空字符结尾的字符数组进行比较
pos1, n1, cp, n2	将 s 中从 pos1 开始的 n1 个字符与指针 cp 指向的地址开始的 n2 个字符进行比较

9.5.5 数值转换

字符串中常常包含表示数值的字符。例如, 我们用两个字符的 string 表示数值 15——字符 '1' 后跟字符 '5'。一般情况, 一个数的字符表示不同于其数值。数值 15 如果保存为 16 位的 short 类型, 则其二进制位模式为 0000000000001111, 而字符串 "15" 存为两个 Latin-1 编码的 char, 二进制位模式为 0011000100110101。第一个字节表示字符 '1', 其八进制值为 061, 第二个字节表示 '5', 其 Latin-1 编码为八进制值 065。

新标准引入了多个函数, 可以实现数值数据与标准库 string 之间的转换:

```
int i = 42;
string s = to_string(i); // 将整数 i 转换为字符表示形式
double d = stod(s);      // 将字符串 s 转换为浮点数
```



367



368 此例中我们调用 `to_string` 将 42 转换为其对应的 `string` 表示，然后调用 `stod` 将此 `string` 转换为浮点值。

要转换为数值的 `string` 中第一个非空白符必须是数值中可能出现的字符：

```
string s2 = "pi = 3.14";
// 转换 s 中以数字开始的第一个子串，结果 d = 3.14
d = stod(s2.substr(s2.find_first_of("+-.0123456789")));
```

在这个 `stod` 调用中，我们调用了 `find_first_of`（参见 9.5.3 节，第 325 页）来获得 `s` 中第一个可能是数值的一部分的字符的位置。我们将 `s` 中从此位置开始的子串传递给 `stod`。`stod` 函数读取此参数，处理其中的字符，直至遇到不可能是数值的一部分的字符。然后它就将找到的这个数值的字符串表示形式转换为对应的双精度浮点值。

`string` 参数中第一个非空白符必须是符号（+ 或 -）或数字。它可以以 `0x` 或 `0X` 开头来表示十六进制数。对那些将字符串转换为浮点值的函数，`string` 参数也可以以小数点（.）开头，并可以包含 `e` 或 `E` 来表示指数部分。对于那些将字符串转换为整型值的函数，根据基数不同，`string` 参数可以包含字母字符，对应大于数字 9 的数。



如果 `string` 不能转换为一个数值，这些函数抛出一个 `invalid_argument` 异常（参见 5.6 节，第 173 页）。如果转换得到的数值无法用任何类型来表示，则抛出一个 `out_of_range` 异常。

表 9.16: `string` 和数值之间的转换

<code>to_string(val)</code>	一组重载函数，返回数值 <code>val</code> 的 <code>string</code> 表示。 <code>val</code> 可以是任何算术类型（参见 2.1.1 节，第 30 页）。对每个浮点类型和 <code>int</code> 或更大的整型，都有相应版本的 <code>to_string</code> 。与往常一样，小整型会被提升（参见 4.11.1 节，第 142 页）
<code>stoi(s, p, b)</code>	返回 <code>s</code> 的起始子串（表示整数内容）的数值，返回值类型分别是 <code>int</code> 、
<code>stol(s, p, b)</code>	<code>long</code> 、 <code>unsigned long</code> 、 <code>long long</code> 、 <code>unsigned long long</code> 。 <code>b</code>
<code>stoul(s, p, b)</code>	表示转换所用的基数，默认值为 10。 <code>p</code> 是 <code>size_t</code> 指针，用来保存
<code>stoll(s, p, b)</code>	<code>s</code> 中第一个非数值字符的下标， <code>p</code> 默认为 0，即，函数不保存下标
<code>stoull(s, p, b)</code>	
<code>stof(s, p)</code>	返回 <code>s</code> 的起始子串（表示浮点数内容）的数值，返回值类型分别是
<code>stod(s, p)</code>	<code>float</code> 、 <code>double</code> 或 <code>long double</code> 。参数 <code>p</code> 的作用与整数转换函数
<code>stold(s, p)</code>	中一样

9.5.5 节练习

练习 9.50: 编写程序处理一个 `vector<string>`，其元素都表示整型值。计算 `vector` 中所有元素之和。修改程序，使之计算表示浮点值的 `string` 之和。

练习 9.51: 设计一个类，它有三个 `unsigned` 成员，分别表示年、月和日。为其编写构造函数，接受一个表示日期的 `string` 参数。你的构造函数应该能处理不同数据格式，如 `January 1, 1900`、`1/1/1900`、`Jan 1 1900` 等。

9.6 容器适配器

除了顺序容器外，标准库还定义了三个顺序容器适配器：stack、queue 和 priority_queue。适配器 (adaptor) 是标准库中的一个通用概念。容器、迭代器和函数都有适配器。本质上，一个适配器是一种机制，能使某种事物的行为看起来像另外一种事物一样。一个容器适配器接受一种已有的容器类型，使其行为看起来像一种不同的类型。例如，stack 适配器接受一个顺序容器（除 array 或 forward_list 外），并使其操作起来像一个 stack 一样。表 9.17 列出了所有容器适配器都支持的操作和类型。

369



表 9.17: 所有容器适配器都支持的操作和类型	
size_type	一种类型，足以保存当前类型的最大对象的大小
value_type	元素类型
container_type	实现适配器的底层容器类型
A a;	创建一个名为 a 的空适配器
A a(c);	创建一个名为 a 的适配器，带有容器 c 的一个拷贝
关系运算符	每个适配器都支持所有关系运算符： <code>==</code> 、 <code>!=</code> 、 <code><</code> 、 <code><=</code> 、 <code>></code> 和 <code>>=</code> 这些运算符返回底层容器的比较结果
a.empty()	若 a 包含任何元素，返回 false，否则返回 true
a.size()	返回 a 中的元素数目
swap(a,b)	交换 a 和 b 的内容，a 和 b 必须有相同类型，包括底层容器类型也必须相同
a.swap(b)	

定义一个适配器

每个适配器都定义两个构造函数：默认构造函数创建一个空对象，接受一个容器的构造函数拷贝该容器来初始化适配器。例如，假定 `deq` 是一个 `deque<int>`，我们可以用 `deq` 来初始化一个新的 `stack`，如下所示：

```
stack<int> stk(deq); // 从 deq 拷贝元素到 stk
```

默认情况下，stack 和 queue 是基于 deque 实现的，priority_queue 是在 vector 之上实现的。我们可以在创建一个适配器时将一个命名的顺序容器作为第二个类型参数，来重载默认容器类型。

```
// 在 vector 上实现的空栈
stack<string, vector<string>> str_stk;
// str_stk2 在 vector 上实现，初始化时保存 svec 的拷贝
stack<string, vector<string>> str_stk2(svec);
```

对于一个给定的适配器，可以使用哪些容器是有限制的。所有适配器都要求容器具有添加和删除元素的能力。因此，适配器不能构造在 array 之上。类似的，我们也不能用 forward_list 来构造适配器，因为所有适配器都要求容器具有添加、删除以及访问尾元素的能力。stack 只要求 push_back、pop_back 和 back 操作，因此可以使用除 array 和 forward_list 之外的任何容器类型来构造 stack。queue 适配器要求 back、push_back、front 和 push_front，因此它可以构造于 list 或 deque 之上，但不能基于 vector 构造。priority_queue 除了 front、push_back 和 pop_back 操作之外还要求随机访问能力，因此它可以构造于 vector 或 deque 之上，但不能基于 list 构造。

370